INFO 7375 — Prompt Engineering for Generative AI

Summer 2026 | Northeastern University — College of Engineering

> **Student Project Guide: Step-by-Step Roadmap for Building Your AI Application**

Project Overview

You will design, build, test, secure, and present a functional AI application powered by LLMs. The project runs across all 14 weeks of the semester, with 8 graded milestones tied to homework assignments and a live final presentation.

Weight	25% of final grade
Milestones	8 graded milestones
HW Submissions	5 assignments
Final	Live demo presentation + final report

---

1. Choosing Your Project (Week 1)

The very first step — before any code — is picking a project domain that genuinely interests you and identifies a real-world problem worth solving. Two tracks are available:

Tech-Focused Track

Project Idea	Core Complexity
Knowledge Base Assistant (RAG App)	RAG pipeline, embeddings, indexing, guardrails
Compliance Checker / Policy Analyzer	Structured outputs, RAG accuracy tuning, safety constraints

Meeting Analyst (Summaries + Actions + Risks)	Multi-output tasks, transcription + analysis
Legal Document Simplifier & Clause Reviewer	Structured generation, multi-step analysis
Customer Support Agent (Semi-Autonomous)	Classification + generation + workflow automation

Consumer-Friendly Track

Project Idea	Core Complexity
AI Study Coach / Learning Companion	Content generation, quiz creation, spaced-repetition automation
Personal Wellness & Habit Coach	Multi-step reasoning, journaling memory, dashboards
Travel Planner & Itinerary Builder	Planning chains, multi-step tasks, export workflows
Smart Meal Planner & Grocery Assistant	Multi-output generation + structured lists + recipe API integrations
Fitness & Workout Generator	Rule-based + AI hybrid logic, tracking dashboard

How to Choose

- Pick a domain you can explain in one sentence: *"A tool that does X for Y users."*
- Confirm it requires LLM integration — not just a database query or simple rule engine.
- Think about Week 7 early: your project must connect to an external knowledge source via RAG.
- Check that your idea can grow — later milestones add memory, agents, security, and evaluation.

---

2. The 8-Milestone Roadmap

#	Milestone	Week(s)	Deliverable	Key Focus

M1	Project Proposal	1–2	HW1	Define domain, problem, solution approach, and use cases
M2	Basic Prototype	3–4	HW2 (System Prompt)	First API call + complete system prompt responding to your core user request
M3	RAG & Memory	7–8	HW3	External knowledge source via RAG + memory built into the app
M4	Tools & Automation	9–10	HW4	Add agents/tools, LangChain/Bedrock pipeline, automated workflow
M5	Evals & Security	11–12	HW5	Test cases, LLM-as-a-judge eval, prompt injection defense
M6	Ethics & Responsible Deployment	13	In-lab	Bias audit, privacy check, ethical guidelines added to prompts
M7	Final Presentation	14	Live in-class	Full demo: working app + all prompt engineering decisions explained
M8	Final Report	14	Submission	Full documentation: process, challenges, solutions, Jupyter Notebook

---

3. Week-by-Week Build Guide

Weeks 1–2: Foundation & Proposal

Goal: Choose your domain and draft your project proposal (HW1).

- Write a one-paragraph problem statement: who is the user, what problem do they have, how does AI help?
- Identify the NLP task type at the core of your project (summarization, classification, generation, Q&A, etc.).
- Draft 3–5 use cases showing how a real user would interact with your tool.
- Sketch a high-level architecture: inputs → LLM → outputs.

- **HW1 Submission:** Project Proposal with problem statement, use cases, and architecture sketch.

Weeks 3–4: First API Call & System Prompt

Goal: Stand up your Jupyter Notebook and make your first real API call (HW2).

- Set up your Jupyter Notebook and install the OpenAI (or chosen) SDK.
- Make the first API call for your core user request — even a rough response counts.
- Week 4 lab: write your complete system prompt. Define the persona, tone, constraints, and output format.
- **HW2 Submission:** Working system prompt with annotated explanation of each section.
- **Tip:** Use role grounding (*"You are an expert..."*) and specify output format explicitly (JSON, markdown, etc.).

Weeks 5–6: Reasoning, Decomposition & Prompt Stability

Goal: Apply advanced prompting techniques and stress-test your prompts.

- Add chain-of-thought or task decomposition to your system prompt where reasoning is needed.
- Apply few-shot examples if your output format needs to be very consistent.
- Run a sensitivity test: change one word in your prompt and observe output drift.
- Fine-tune tone and format based on results. Document what changed and why.
- No new HW this week — but this work directly improves your M3/HW3 quality.

Weeks 7–8: RAG & Memory

Goal: Ground your app in real data and give it memory (HW3).

- Choose an external knowledge source: PDFs, web pages, a database, or an API.
- Implement a basic RAG pipeline: chunk documents → embed → store in vector DB → retrieve on query.
- Add grounding: your LLM responses should cite or reference retrieved content.
- Build memory into your app: at minimum, maintain conversation history so context carries across turns.

- **HW3 Submission:** RAG pipeline working end-to-end + memory demonstrating stateful conversation.

Weeks 9–10: Agents, Tools & Automation

Goal: Give your app the ability to take actions, not just generate text (HW4).

- Identify tools your app needs (web search, calculator, database lookup, calendar, API calls).
- Implement the ReAct pattern: model reasons → calls a tool → observes output → continues.
- Explore LangChain or Azure Prompt Flow / AWS Bedrock to automate your pipeline.
- Build at least one automated workflow where a user query triggers a multi-step chain without manual intervention.
- **HW4 Submission:** Agent with tool use + automated pipeline demonstrated end-to-end.

Weeks 11–12: Testing, Evaluation & Security

Goal: Evaluate quality rigorously and harden your app against attacks (HW5).

- Write 10+ functional test cases covering happy paths and edge cases.
- Use LLM-as-a-judge: have a model evaluate your app's responses for relevance, faithfulness, and clarity.
- Attempt prompt injection on your own app — try to break it, then fix the vulnerabilities.
- Add defensive system prompt guidelines: input validation, output safeguards, refusal behavior.
- **HW5 Submission:** Evaluation report + before/after security hardening demonstration.

Week 13: Ethics, Bias & Responsible Deployment

Goal: Audit your app for fairness and add ethical guardrails.

- Run a demographic bias test: does your app respond differently based on names, genders, or nationalities?
- Add a privacy check: ensure no PII is stored in memory or logs unnecessarily.

- Embed ethical guidelines into your system prompt (scope limits, escalation behavior, refusal cases).
- Document your responsible deployment decisions for inclusion in the final report.

Week 14: Final Presentation, Demo & Report

Goal: Showcase everything you've built and document it thoroughly.

- **Live demo:** walk through a real user interaction, explaining each prompt engineering decision made.
- Presentation must cover: problem, architecture, prompting techniques used, evaluation results, and security measures.
- **Final Report** must include: development process narrative, challenges faced, solutions implemented, and all code.
- Submit your Jupyter Notebook with clean, commented code alongside the written report.

---

4. How Your Project is Graded (25% of Final Grade)

The project is assessed across five equally-weighted categories (5% each). Every category has 5 sub-criteria worth 1 point each, totaling 25 points.

Prompt Engineering Concepts (5%)

Sub-Criterion	Description
Basic Techniques	Effective use of basic prompt engineering techniques
Advanced Techniques	Application of few-shot learning or chain-of-thought prompting
Conceptual Understanding	Demonstrates deep understanding of prompt engineering principles
Integration	Seamless integration of prompt engineering into app functionality
Innovation in Prompting	Creative and effective use of prompts to achieve desired outcomes

Quality & Functionality of the Tool (5%)

Sub-Criterion	Description
Performance	App performs its intended functions accurately and efficiently
Usability	Demo is user-friendly and easy to navigate
Reliability	App operates without crashes or significant bugs
Scalability	App can handle increased load or additional features
User Feedback	Positive feedback from users or testers regarding functionality

Iterative Development & Peer Review (5%)

Sub-Criterion	Description
Participation	Active involvement in all stages of the development process
Feedback Incorporation	Effectively incorporates feedback from users, peers, and instructors
Iteration	Demonstrates iterative improvement based on testing and feedback
Collaboration	Works well with peers during the review and development process
Documentation	Maintains clear and thorough documentation of development stages

Final Presentation Clarity (5%)

Sub-Criterion	Description
Organization	Presentation is well-structured and logically organized
Clarity	Information is presented clearly and concisely
Comprehensiveness	Covers objectives, methods, results, and conclusions
Visual Aids	Effective use of slides and diagrams to enhance understanding
Engagement	Presentation holds the audience's attention

Final Report Thoroughness (5%)

Sub-Criterion	Description
Structure	Report is well-structured and logically organized
Clarity	Information is presented clearly and concisely
Comprehensiveness	Covers all aspects including objectives, methods, results, conclusions
Format	Effective use of fonts, diagrams, white space, and grammar
Engagement	Report is engaging and readable

---

5. Practical Tips for Success

Start narrow, then expand.
A focused tool that does one thing well scores higher than an ambitious tool that half-works. Nail your core use case first (Weeks 1–4), then add complexity.

Document as you go.
The final report is much easier if you keep a running dev log. After each lab or HW, write 3–5 bullets: what you tried, what broke, and what you changed.

Version your prompts.
Save each major version of your system prompt in a separate notebook cell. Graders reward visible iteration — showing v1 vs v3 with a rationale is strong evidence of mastery.

Test with adversarial inputs early.
Don't wait until Week 12 to try breaking your app. If you test from Week 4 onward, your security milestone will be much less stressful.

Use peer review seriously.
Peer review is worth 5% of your final grade separately. More importantly, your classmates will catch UX issues and edge cases you've become blind to.

Cite your retrieval sources in responses.
For RAG apps, have your LLM include a "Sources:" section in every response. This single change dramatically improves both trustworthiness and your evaluation score on faithfulness.

Communicate with the TA before deadlines.
Per the late work policy, work submitted late without prior communication will not be graded. If you're falling behind, reach out proactively.

Instructor: Prof. Shirali Patel — shi.patel@northeastern.edu